

Machine Learning Assignment 1

Documentation:

SID:

22035587

Name:

Manan Bhatia

How to run the code:

For each question of the given 2 tasks, select the given codes which is related to the question and run the code with this icon in Spyder:



Description:

Overall, the solutions that I have given should correspond to the code that I put by referring to the lecture notes and tutorials. The screenshots of the answers are given in this document. And the given outputs should be correct with the given codes.

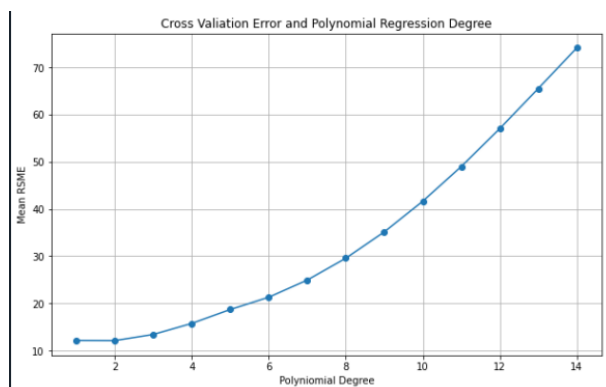
Outputs:

Task 1 Q2:

```

Degree 1 : Mean RSME = 12.17
Degree 2 : Mean RSME = 12.13
Degree 3 : Mean RSME = 13.44
Degree 4 : Mean RSME = 15.77
Degree 5 : Mean RSME = 18.72
Degree 6 : Mean RSME = 21.32
Degree 7 : Mean RSME = 24.96
Degree 8 : Mean RSME = 29.57
Degree 9 : Mean RSME = 35.15
Degree 10 : Mean RSME = 41.64
Degree 11 : Mean RSME = 48.98
Degree 12 : Mean RSME = 57.01
Degree 13 : Mean RSME = 65.50
Degree 14 : Mean RSME = 74.09

```



Q3:

```

...: print(f"Coefficients for the Best Degree {bestdegree} Model is: {bestmodel.coef_}")
...: print(bestmodel.coef_)
...: print(bestmodel.intercept_)
Coefficients for the Best Degree 2 Model is: [[ 0.          -1.54847161  0.00394948]]
[[ 0.          -1.54847161  0.00394948]]
[278.56036242]

```

Q4:

```
Coefficients:
AGE: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
      -0.0018498  0.20381035]
ED-LEVEL: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
            -0.0018498  0.20381035]
SMOKING STATUS: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
                 -0.0018498  0.20381035]
EXERCISE: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
            -0.0018498  0.20381035]
WEIGHT: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
          -0.0018498  0.20381035]
SERUM-CHOL: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
              -0.0018498  0.20381035]
IQ: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
      -0.0018498  0.20381035]
SODIUM: [ 0.4423102 -1.13252925 -0.06743506 -1.01879418  0.13386439  0.00439491
          -0.0018498  0.20381035]
Intercept: 66.68546356952064
Mean RMSE: 7.884396356603955
```

Q5:

```
Coefficients:
AGE: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
      -0.0020778  0.20372363]
ED-LEVEL: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
            -0.0020778  0.20372363]
SMOKING STATUS: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
                 -0.0020778  0.20372363]
EXERCISE: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
            -0.0020778  0.20372363]
WEIGHT: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
          -0.0020778  0.20372363]
SERUM-CHOL: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
              -0.0020778  0.20372363]
IQ: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
      -0.0020778  0.20372363]
SODIUM: [ 0.44238416 -1.12823231 -0.0676081 -1.0175469  0.13384909  0.00443404
          -0.0020778  0.20372363]
Intercept: 66.70686764934337
Mean RMSE: 7.884127060280879
```

Task 2 Q2:

```
In [69]: from sklearn.decomposition import PCA
...: ratio = 0.88
...: pca = PCA(n_components=ratio, svd_solver='full')
...: Xpca = pca.fit_transform(X)
...: Xtrainpca = pca.fit_transform(Xtrain_scale)
...: Xtestpca = pca.transform(Xtest_scale)
...:
...: selectedcomp = pca.n_components_
...: variance = sum(pca.explained_variance_ratio_)
...:
...: print(f"The variance of the PCA is: {variance}")
The variance of the PCA is: 0.88056752824955
```

Q3:

```
In [73]: X_train2, X_test2, y_train2, y_test2 = train_test_split(Xpca, y, test_size=0.2, random_state=42)
...: print(f"X_train = {X_train2}")
...: print(f"X_test: {X_test2}")
...: print(f"y_train: {y_train2}")
...: print(f"y_test: {y_test2}")
X_train = [[ -0.63708201 -3.64186565  3.53017775 ...  1.80139181 -0.41202189
-0.21893259]
 [ -2.38987146 -3.36635404 -6.13863947 ... -0.27409406  2.11657876
 1.4721137 ]
 [ -3.5875928  4.71857497 -0.03002456 ...  0.11315007 -0.51791457
 1.0601425 ]
 ...
 [ -8.36149233 -1.20175818  2.08869722 ...  0.61440896  0.27236028
-0.03546693]
 [ 10.48732774 -9.04332913 -2.26014232 ...  0.16307098  0.43929635
-0.24111619]
 [ 10.58446907 -12.40620907  2.7963609 ...  0.26522847  0.56816083
-0.81298149]]
X_test: [[ 4.99068839 -1.66773491  0.83715366 ... -0.38790033 -0.09992748
-1.05311858]
 [ -3.08114176  4.27469627 -3.65278884 ...  0.23936086 -0.01393142
 0.5137962 ]
 [ -0.43696746  7.83599812  3.14022109 ...  0.24162283 -0.43580663
 0.777304 ]
 ...
 [ -3.34146153 -0.95419798  7.69857515 ... -0.12656007  0.01624954
-0.21525537]
 [ -5.45084919  5.94520558 -8.33313808 ... -0.07343864  0.91113238
-0.25936426]
 [ -8.16399385 -2.19457415  3.66981812 ... -0.21592543 -0.25586474
-0.38872489]]
y_train: [0 0 0 ... 0 0 0]
y_test: [0 0 0 ... 0 0 0]
```

Q5:

```
In [74]: ytrain_predict = model.predict(X_train2)
...: ytest_predict = model.predict(X_test2)
...:
...: print(f"The prediction is {ytest_predict}")
...: print(f"The prediction is {ytrain_predict}")
The prediction is [0 0 0 ... 0 0 0]
The prediction is [0 0 0 ... 0 0 0]
```

Q6:

```
....: print(f"The Accuracy test is: {accuracytest}")
....: print(f"The Accuracy train is: {accuracytrain}")
....: print(f"The Confusion Matrix is: {confused}")
....: print(f"The Classification Report is: {classificationtest}")
....: print(f"Number of Misclassified Digits: {misclassified}")
The Accuracy test is: 0.9877857142857143
The Accuracy train is: 0.9885178571428571
The Confusion Matrix is: [[12524   80]
 [  91 1305]]
The Classification Report is:      |      precision    recall  f1-score   support

      0      0.99      0.99      0.99     12604
      1      0.94      0.93      0.94     1396

   accuracy                0.99     14000
  macro avg              0.97      0.96      0.97     14000
 weighted avg            0.99      0.99      0.99     14000

Number of Misclassified Digits: [  40   53   78  284  311  349  362  420  495  579  651  686
   750   755   870  1068  1091  1301  1364  1409  1450  1481  1522  1547
  1695  1790  1873  2183  2209  2255  2387  2391  2407  2551  2619  2625
  2666  2737  2759  2806  2972  2989  3164  3349  3395  3404  3446  3731
  3805  3987  4011  4050  4152  4198  4323  4501  4657  4809  4944  4981
  5042  5114  5180  5209  5262  5276  5322  5331  5396  5439  5592  5603
  5708  5808  5887  5927  5951  5957  5971  6137  6206  6228  6271  6338
  6353  6519  6657  6695  6828  6877  7002  7082  7090  7124  7192  7296
  7297  7314  7486  7502  7689  7998  8090  8104  8136  8140  8141  8164
  8234  8274  8306  8315  8479  8489  8497  8574  8586  8595  8663  8687
  8798  8905  9056  9077  9093  9215  9238  9458  9503  9741  9934  9975
 10024 10025 10179 10203 10227 10464 10481 10640 10663 10844 10929 10938
 11046 11047 11354 11446 11667 11753 11949 12206 12370 12444 12524 12771
 12788 12833 12852 12935 13016 13076 13147 13213 13236 13248 13417 13674
 13807 13973 13975]
```

Answers to Q3 and Q6 from Task 1 & Q7 from Task 2:

Q3: The best degree is 2 because it shows after that degree, it is overfitting whilst degree 2 is optimal.

Q6: To select the best model, it is best to look at the mean rmse of all the models and the lowest one is considered the best model. So therefore, the best model is the ridge regression.

```
In [152]: print(f"The mean of the Best Degree which is
{bestdegree} is: {bestsme}")
The mean of the Best Degree which is 2 is: 12.547523655249586

In [153]: print(f"Mean RMSE for Multiple Linear Regression:
{mean_rsme_value2}")
Mean RMSE for Multiple Linear Regression: 7.884396356603955

In [154]: print(f"Mean RMSE for Ridge Regression:
{mean_rsme_value3}")
Mean RMSE for Ridge Regression: 7.884127060280879
```

Q7: To check if the model is overfit, underfit or good fit, we need to see the accuracy of both the training and test accuracies. If the model is overfit, this means that the model performs well on the training data but isn't generalised well in predicting new data. If the model is underfit, this means the model is too simple to learn the underlying structure of the data.

```
In [156]: print(f"The Accuracy test is: {accuracytest}")
The Accuracy test is: 0.9877857142857143

In [157]: print(f"The Accuracy train is: {accuracytrain}")
The Accuracy train is: 0.9885178571428571
```

The accuracies of both train and test seem to have numbers that are really close to each other, this indicates that the model is a good fit.